

TLP:CLEAR

The PDF viewer that lives in the registry

An HTA mailer aimed at Russian organisations under the guise of an outgoing letter: analysis of a sample from 1 October 2026

Attachment-based mailer | HTA / JScript | Registry-resident loader | Browser credential theft

Parameter	Value
Period of events	1 – 7 October 2026 (UTC)
Date of analysis	7 October 2026
Classification	TLP:CLEAR
Threat level	MEDIUM
Threat type	Malicious attachment: a loader with remote control and browser data theft
Family	Not attributed to any known family by any of the three sample repositories consulted
Infrastructure status	Primary command server answering as of 7 October; the two fallback addresses from the configuration do not respond

1. In short

On 1 October 2026 a file named «ICX № 134ОП-14 от 01.10.2026.hta» went out to Russian organisations. The name imitates an outgoing letter with the reference number and date that a registry office assigns. Antivirus telemetry counted roughly a thousand encounters with the file in the first day, so this is a mass mailing rather than a targeted one. The sample surfaced in public collections on 2 October; none of the three sample repositories we consulted attributed it to a known family.

The file is an HTML application. When opened it draws the interface of a PDF viewer: a dark toolbar with buttons, as in a browser. The user sees a document that "did not open"; meanwhile registry keys have been written in their profile, hidden files created, and a scheduled task registered that brings the loader back every five minutes.

Three things about this sample deserve separate attention.

- The code is stored in the registry, not on disk. Later stages are written under HKCU\Software\RememberMilk as strings of two thousand characters and read back on every launch. Only two short launcher scripts remain on disk.
- The first thing the loader does after establishing persistence is to extract the master key of Chromium-based browsers from the Local State file and decrypt it with Windows means. That key unlocks every saved password and cookie.
- The command servers are configured as a redundant triple, all three in one network of a small hosting company that a year earlier carried RAT distribution sites. Commands arrive as XML and are executed as arbitrary code, so this is not merely a stealer but a full remote access tool.

2. What the file looks like

The .hta extension stands for HTML Application — a page that Windows runs not in a browser but in the mshta.exe process with the rights of an ordinary program. To the recipient it looks like a document: a file name with a number and a date, an icon, and after a double-click a window with a dark toolbar, navigation arrows and a page-number field. The window title, however, reads "Test2": the author did not bother to change it.

The visible part of the page is decoration only. The entire payload sits in eight hidden text blocks that are never displayed. Each block contains strings resembling GUID identifiers: groups of hexadecimal digits separated by hyphens, the groups separated by commas. A ninth hidden block holds a short decoder.

On page load a short VBScript collects the text of all hidden blocks, strips whitespace and line breaks, and hands the result to the MSScriptControl component as a JScript program. The decoder reads the "GUID" strings two characters at a time, converts each pair to a byte, skips zero bytes, and obtains 78 thousand characters of source code. This is how the file evades checks that look for suspicious commands in HTA files in the clear: the page text contains not a single call to a system object.

The code itself resists casual reading: every function and variable name is a random run of eight upper-case letters, and roughly forty percent of the volume consists of empty comments with random content interleaved with every line. Once those are removed, 66 thousand characters of clear structure remain.

3. What happens on the machine

3.1. Persistence

The loader creates three directory trees in the user profile with names that resemble traces of real software: Real\NCix\Pointer, CubaseLite\Loop\Unsafe\Time\Scroll\Separate and ObsidianX\Item\Stop\Link\Export\

Paste. Two scripts with innocuous names, `session_history.js` and `spreadsheet.js`, are placed there, both marked hidden.

Launch is secured twice. The `RunOnce` key receives entries named `Blender3D`, `CloneZillaX` and `Chromix`: on each run every launcher re-registers itself under its own name. In parallel, the `schtasks` command registers three tasks with nested names — `FreshDesker\lp\Connected\Private` (every five minutes), `\Public` and `\Previous` (every fifteen), all with limited rights. The nested name is a trick that keeps the tasks off the first screen of the Task Scheduler. Six launcher scripts end up spread across two directory trees, and deleting some of them does not stop the infection: the survivors restore the rest.

3.2. Code in the registry

Both scripts on disk are short and identical in purpose: read a string from the registry, reconstruct a program from it, execute it. The program itself lives under `HKCU\Software\RememberMilk` and four subkeys. Because a registry string value is limited in length, the code is cut into pieces of two thousand characters and written under names of the form `<name>_0`, `<name>_1` and so on; on reading, the pieces are joined back together. Execution goes through the Function constructor — an equivalent of `eval` that attracts less attention.

The practical consequence for incident response: an antivirus scan of the disk after the HTA file has been deleted will find two harmless-looking one-kilobyte scripts and nothing else. The infection lives in the registry.

3.3. Browser key theft

One of the loader functions assembles and runs a PowerShell command that reads the Local State file from a Chromium-based browser profile, takes the `os_crypt.encrypted_key` field, discards a five-byte prefix and decrypts the remainder through `ProtectedData::Unprotect` in the current user context. The result — the master key with which the browser encrypts saved passwords and cookies — is written to a file for exfiltration. The method is standard and well documented; what is unusual is that it is driven from JScript rather than from a compiled stealer.

Next to it are directory traversal functions that build a name of the form `"date_time_size_name"` for every file — groundwork for collecting documents on command from the server.

3.4. Talking to the server

We observed the exchange ourselves, in our own sandbox: requests were captured at the moment of encryption, before they went out, so they are visible in full despite TLS.

First the machine sends a GET to the path `/sts`. This is a liveness probe and carries nothing identifying. The reply is exactly ten bytes, identical on 2 and on 7 October: a Russian obscenity written in Latin letters. A trifle, but it says something about the author.

Then comes a POST to a `/vpr-` path with a random suffix of 7 to 12 characters that the client invents afresh for every request — and this is where the substance is. The request carries three headers of its own: `C-I` with the machine identifier (the prefix `4tg-` and eight random characters), `S-K` with a sixteen-digit hexadecimal key, and `R-T` with the word `conf`, meaning "I am asking for configuration". The body is empty and the content type is declared as XML.

The value of S-K turned out to be constant. We ran the sample twice, on a clean system, with different machine identifiers — and the key was the same both times. So it is not a session key but a build marker: it will outlive a change of addresses and tie future waves to this same batch. For defenders that is worth more than any IP address, and the triple C-I, S-K and R-T in a single request does not occur in ordinary traffic.

The loader parses server responses as XML with the MSXML2.DOMDocument component, extracts the content of the required tag and passes it to the Function constructor. The result is returned as a string "SUCCESS|..." or "ERROR|...". In other words, the server can execute arbitrary JScript on the machine; browser key theft and file enumeration are merely what was prepared in advance.

4. Infrastructure

All three addresses belong to a single autonomous system, AS197170; the network is registered to a company with an address in the Seychelles and equipment in the Netherlands. On the third address, 45.156.87.3, passive DNS shows that in October 2025 a domain whose name directly referenced the XWorm RAT was hosted, followed later by several more domains seen in malware distribution. The hosting provider, then, is not a random choice.

The primary address 45.156.87.120 is still working on 7 October: port 80 redirects to HTTPS, port 443 answers with nginx 1.24.0 on Ubuntu, the root returns 404, /sts answers, and a GET to /vpr- is refused with 403 — the path accepts POST only. The certificate is self-signed, issued to its own IP address on 24 March 2026: the infrastructure was prepared half a year before this mailing and probably served earlier waves as well. The two fallback addresses from the configuration, by contrast, answer on neither port 80 nor 443 — checked from several different exit points.

No other samples contacting these addresses were found in public collections. Either this is the first wave on this infrastructure, or earlier waves escaped notice.

5. What the detonation showed

We ran the sample in our own isolated environment, with all of its outbound traffic routed through an anonymising network so as not to reveal where the observation is being made from.

The first attempt produced nothing: a script error dialog and that was all. The reason turned out to be instructive. The MSScriptControl component, through which the file executes its payload, exists only in a 32-bit build. By default the sandbox launches the 64-bit version of mshta, and there the call to the component fails. When a recipient opens the attachment with a double click, Windows launches the 32-bit version — and everything works. Hence a practical conclusion: examining such a file under a 64-bit host yields the false verdict that nothing happens. We added a launch through the 32-bit host, and the sample came alive.

From there everything followed the scheme described above: three scheduled tasks, six launcher scripts, two attempts to extract the master key — separately for Chrome and for Edge — and one exchange with the server. The server replied: forty bytes to the configuration request.

And then nothing happened. Over seven minutes of observation the loader wrote not a single module to the registry, created no new files and never contacted the server again. So the command node is alive and answering, but it issued our machine no work. Why, we cannot say with confidence. Two explanations are

possible: the operator selects victims by the origin of their traffic, and we arrived from an anonymising network rather than from Russian corporate mail; or the delivery of modules in this wave has already been wound down. We cannot tell these apart, and we will not present a guess as a fact.

6. Indicators

Addresses and registry keys are given exactly as extracted from the code. Server addresses and file hashes are suitable for blocking; registry keys, task names and paths are for finding machines that are already infected.

Type	Value	Note
Command server	45[.]156[.]87[.]120:443	Primary; answering as of 7 October. nginx 1.24.0 (Ubuntu), self-signed certificate CN=45[.]156[.]87[.]120 issued 2026-03-24
Command server	176[.]65[.]132[.]239:443	Fallback from the configuration; no connection as of 7 October
Command server	45[.]156[.]87[.]3:443	Fallback; unresponsive. In 2025 the address served RAT distribution domains
URL	hxxps://45[.]156[.]87[.]120/sts	Liveness probe. The response is a constant 10 bytes
URL pattern	hxxps://<server>/vpr-<7-12 characters>	POST for configuration; the suffix is generated afresh by the client on every request
HTTP header	C-I: 4tg-XXXXXXXXX	Identifier of the infected machine
HTTP header	S-K: D9F72A4E6A3D4FA6	Constant: identical across separate infections, so a build key rather than a session key
HTTP header	R-T: conf	Request type. The triple C-I + S-K + R-T in one POST is the primary network marker
User-Agent	Mozilla/4.0 (compatible; Win32; WinHttp.WinHttpRequest.5)	Default value of the WinHttpRequest COM object
SHA-256	1d79ab0c2d0183fca2772b008c4f146ec0c83000865a1bb2be2c8209d0abb88	HTA file, 190,084 bytes
MD5	66c671e994bd7d61a1b1811ed46a5dbe	Same file

Filename	ICX № 134ОП-14 от 01.10.2026.hta	Lure styled as an outgoing letter reference number
Registry key	HKCU\Software\RememberMilk	Storage for later stages, strings of 2,000 characters
Registry key	HKCU\...\CurrentVersion\RunOnce\Blender3D	Autostart: wscript.exe ...\ session_history.js
Registry key	HKCU\...\CurrentVersion\RunOnce\CloneZillaX	Autostart: wscript.exe ...\ backup_archive.js
Registry key	HKCU\...\CurrentVersion\RunOnce\Chromix	Autostart: wscript.exe ...\ email_list.js
Scheduled task	FreshDesker\lp\Connected\Private	Every 5 minutes: wscript.exe ...\ ObsidianX\...\ spreadsheet.js
Scheduled task	FreshDesker\lp\Connected\Public	Every 15 minutes: wscript.exe ...\ ObsidianX\...\ download_queue.js
Scheduled task	FreshDesker\lp\Connected\Previous	Every 15 minutes: wscript.exe ...\ ObsidianX\...\ screen_map.js
Path	%LOCALAPPDATA%\CubaseLite\Loop\Unsafe\...	Hidden launchers: session_history.js, backup_archive.js, email_list.js
Path	%LOCALAPPDATA%\ObsidianX\Item\Stop\...	Hidden launchers: spreadsheet.js, download_queue.js, screen_map.js
Path	%LOCALAPPDATA%\Real\VCix\Pointer\	Loader working directory
Launcher SHA-256	a690ceebefc4ed804fe3217a25b4ed94786d58d7ee4024f36887aa620ba793be	backup_archive.js, 2,843 bytes
Launcher SHA-256	d3b6c64efadcc722d1feabb677f0e11001901e97f0be35c8fe12731a6dc0ab70	email_list.js, 2,898 bytes
Launcher SHA-256	f43139ef9d88ede77a4bc9b2d44eb442665d3da256077e76dab4b610d664b3ad	session_history.js, 3,534 bytes

7. What to look for on your machines

- A wscript.exe process started from a RunOnce entry or a scheduled task, with a script in %LOCALAPPDATA% inside a directory with a "software-like" name. This does not happen in normal operation.
- A scheduled task with a nested path in its name and a five-minute interval that launches wscript.exe.

- The key HKCU\Software\RememberMilk, and any key holding dozens of string values named <name>_0, _1, _2 — that is code in the registry.
- PowerShell launched from under wscript.exe or mshta.exe that reads the Local State file from a browser profile.
- Creation of the MSScriptControl.ScriptControl object by the mshta.exe process. The component has long fallen out of legitimate use.
- At the perimeter: connections to port 443 of the three addresses in the table, and requests to the paths /sts and /vpr-.
- At the mail gateway: .hta attachments should be blocked unconditionally. The format has no use in business correspondence.

8. What we do not know

We have not seen the message that carried the file. The file name points to the pretext of "an outgoing letter from the organisation", but the body of the covering message, the sender address and the recipient list are unknown to us; the assessment "mass mailing" rests on an antivirus telemetry counter, not on our own observation.

We did not obtain the second-stage modules. The server answers, but to the configuration request it returned forty bytes and nothing more; nothing was written to the registry or to disk. The content of those forty bytes could not be read — our interception truncates its record at a couple of hundred bytes, and the response headers exhaust that limit, while the response itself did not survive in process memory. We therefore know what the loader can do and how it queries the server, but not what the server replies to those it considers a suitable victim.

Family and operator are not established. The combination of traits — an HTA with code in "GUID" strings, stages stored in the registry, nested task names, Chromium key theft from JScript — is characteristic of one lineage of commercial JScript loaders, but we will not put a name to it without comparison against a larger set of samples.

TLP:CLEAR — This document may be distributed without restriction.